Pattern Recognition Letters
journal homepage: www.elsevier.com

A cache in the weights? Dynamic evaluation and temporal locality in small language models

Ruben Tous[a,**], Beatriz Otero[a], Josep Lluis Berral[a]

[a]*Department of Computer Architecture, Universitat Politècnica de Catalunya, Jordi Girona 1-3, 08034 Barcelona, Spain*

ABSTRACT

Dynamic evaluation lets a language model keep learning from the text it reads at inference time, at the cost of forgetting. Because inputs such as books or code bases show temporal locality, this forgetting could act as eviction, making the weights of a small model a cache of the recent context beyond the attention window. We test whether this behaviour arises on its own in a standard transformer taking plain gradient steps as it reads. Small Pythia models read PG-19 books in chunks of 256 tokens, each scored without the preceding text in context, and reading in order is compared with reading the same chunks shuffled. The weights hold a memory of the current book beyond the attention window, stored mostly in the MLPs. It depends on the order of the input and halves within a few thousand tokens of other books. However, it shares the weights with general knowledge: knowledge of the same domain is largely preserved, but out-of-domain text is partly forgotten, and every setting we tried that strengthens the cache also increases this collateral forgetting. On 200k-token books, dynamic evaluation closes about half the gap between a 14M-parameter model and a frozen model twice its size.

© 2026 Elsevier Ltd. All rights reserved.

1. Introduction

Small language models are cheaper to run and can be deployed on memory-constrained devices, but their limited capacity restricts how much knowledge and context they can retain (Liu et al., 2024). Yet what a model reads at inference time often shows temporal locality, i.e. recently seen content is likely to be seen again: a book keeps its characters, places and turns of phrase for hundreds of thousands of tokens, and so can a long conversation or a code base, far longer than an attention window lasts. Memory hierarchies exploit this property with a small cache that holds recently used data and lets older data be pushed out, so that a small memory serves a workload as though it were a large one (Denning, 1968). By analogy, a small model whose parameters cached its current context, instead of permanently holding everything it might ever need, could perform on that context comparably to a larger model.

Dynamic evaluation (Krause et al., 2018, 2019; Rannen-Triki et al., 2024) and test-time training (Feng et al., 2026; Bansal et al., 2026; Tandon et al., 2025) let a language model keep learning from the text it reads, adapting its weights to the current input. The cost is forgetting, the catastrophic interference of sequential training (McCloskey and Cohen, 1989), usually treated as a problem. In a small model, however, locality and a forgetting that spares recent content could together give the behaviour of a cache: the weights would hold the recent context beyond the window, and new material would push out old material.

We ask whether this cache behaviour arises on its own in a standard transformer that takes plain gradient steps as it reads. If it does, three things should be true. The benefit should depend on *locality*: it should be larger when the input stays on the same material than when the same text arrives in a random order. New material should push out old material (*eviction*), the useful side of forgetting. And the cache should leave the knowledge the model already had largely intact; we call the loss of that knowledge, the harmful side, *collateral forgetting*.

We test this with small Pythia models (Biderman et al., 2023) reading PG-19 books (Rae et al., 2020), split into chunks of 256 tokens that are each scored without the previous chunks in context, so that anything that helps from further back must come through the weights. Our contributions are: (i) we measure the reach of the memory and its dependence on input locality, and

**Corresponding author.
e-mail: ruben.tous@upc.edu (Ruben Tous)

compare it with a token-count cache; (ii) we locate where in the network it is stored; (iii) we measure how fast other text evicts it and how much general knowledge, inside and outside the domain, is forgotten; (iv) we study how the update rule and model size shape the cache and its collateral forgetting; and (v) we assess what dynamic evaluation is worth against a larger frozen model on long books.

2. Related work

Cache language models. Text has temporal locality: a word that has occurred in a document is far more likely to recur than its corpus frequency predicts (Church, 2000), and source code is similarly repetitive within a file (Tu et al., 2014). Cache language models exploit this by interpolating the model's prediction with a distribution estimated from the recent history (Kuhn and De Mori, 1990; Jelinek et al., 1991). Neural variants store recent hidden states (Grave et al., 2017; Khandelwal et al., 2020), and several transformers extend the attention window with recurrent, compressed or external memories (Dai et al., 2019; Rae et al., 2020; Wu et al., 2022; Bulatov et al., 2022). In all of these the past is held in activations or a separate store and the parameters stay fixed; we instead restrict the context so that the past can only be held in the parameters.

Dynamic evaluation and test-time training. Dynamic evaluation updates a trained model by gradient descent on the part of the sequence already observed, so that each token is predicted before it is trained on (Mikolov et al., 2010; Graves, 2013). Krause et al. (2018, 2019) motivate it by the recurrence of patterns within a sequence and favour recent history through a decay towards the trained weights, and Rannen-Triki et al. (2024) interpret it as a memory in the weights complementary to the context window and show that, under distribution shift, a smaller model with online learning can offer a better compute–loss trade-off than a larger static one. That dynamic evaluation benefits from locality is thus the premise of this literature; we quantify it by comparing the same text in order and shuffled, and use neither decay nor resets so that eviction can be measured directly. Test-time training adapts a model to each test instance (Sun et al., 2020) and has inspired layers and memories that learn as they read (Sun et al., 2024; Behrouz et al., 2024); recent methods for transformers update the MLP output projections (Feng et al., 2026), the attention queries (Bansal et al., 2026) or the MLPs of a meta-trained model (Tandon et al., 2025), descending from fast weights (Hinton and Plaut, 1987; Schmidhuber, 1992; Ba et al., 2016). They fix by design which parameters adapt and how fast the memory fades; we study an unmodified pretrained model and ask where the memory forms and how long it persists.

Forgetting. Catastrophic interference was first documented in networks trained sequentially (McCloskey and Cohen, 1989; French, 1999) and has motivated methods that protect important parameters (Kirkpatrick et al., 2017). Large language models lose general capabilities during continual fine-tuning (Luo et al., 2023), models updated by dynamic evaluation degrade on older text (Lazaridou et al., 2021), and knowledge from pretraining decays as a power law of later training steps (Chang et al., 2024); much of it appears to reside in the feed-forward layers (Geva et al., 2021; Meng et al., 2022). Forgetting may also be adaptive, reflecting the decreasing probability that an item will be needed again (Anderson and Schooler, 1991), which is the assumption a cache relies on. We measure both the eviction of a book and the loss of general knowledge in the same runs.

3. Setup

A model reads an *input stream*, a single ordered sequence of text, once, in consecutive chunks of 256 tokens. Attention covers only the current chunk, so whatever the model uses from earlier parts of the stream must be carried in its weights. Under dynamic evaluation the model (the *learner*) scores each chunk and then takes one update step on the chunk's mean loss, so every token is predicted before it is trained on; the frozen pretrained model reading the same stream is the reference. We use Pythia-14M, 31M and 70M (Biderman et al., 2023) (6 layers each, same tokenizer), in float32 with seed 0.

Input streams. All streams are built from PG-19 books (Rae et al., 2020), each tokenized on its own and cut into non-overlapping chunks, so that no chunk spans two books; the learner starts from the pretrained weights and is never reset. The *main set* holds 25 test books cut to their first 78 chunks (19,968 tokens), 1,950 chunks in total, arranged in four orders of decreasing locality: book after book ($K = 1$); groups of 5 books interleaved chunk by chunk ($K = 5$); all 25 interleaved ($K = 25$); and a random permutation (shuffled). Any difference between these orders is due to the order alone. The *long* stream holds 5 test books of 100k tokens and the *very long* stream 4 books of 200k tokens, both read book after book. Learning rates are selected on a separate *val-pick* stream of 10 PG-19 validation books of 78 chunks, as the value with the lowest mean loss from a grid of 1 and 3 times powers of ten, extended until both neighbours of the best value have been tried; no reported result uses val-pick. The Pile (Gao et al., 2020), on which Pythia was trained, includes a PG-19 component of 28,602 documents (Biderman et al., 2022), the size of PG-19's training split, so the test books are very likely absent from the training data.

Update rules. *Plain SGD* uses no momentum or weight decay; *Adam* (Kingma and Ba, 2015) starts each stream with a fresh state; and *normalized SGD* moves every tensor by the same relative amount per step, $W \leftarrow W - \eta \|W\| \, g/\|g\|$. All three update all parameters; some plain-SGD runs on Pythia-14M update only a subset (the output embedding, everything else, the MLPs, the attention, or the MLP of one layer), each at its own learning rate.

Snapshots and held-out text. To see what the weights hold at a point of a stream, a frozen copy of the learner (a *snapshot*) scores text outside the stream, while the learner reads on. For collateral forgetting, snapshots are taken every 40 chunks and scored on two held-out sets never trained on: 76 chunks of

the detokenized WikiText-103 test split (Merity et al., 2017), outside the domain, and 6 other PG-19 validation books of 15 chunks, inside it. For eviction, each of 20 main-set books A is split into A1 (40 chunks) and A2 (38 chunks); the stream is A1 followed by I tokens of other PG-19 books, and at its end a snapshot scores A2 and the same span C2 of a control book C that was never read.

Metrics. The *gain* is the frozen loss minus the loss under dynamic evaluation on the same tokens, in nats/token, always against the model's own frozen loss. *Distance bins* group tokens by how far back the same token id last occurred: in the same chunk (visible to attention), in an earlier chunk less than 1k, 1k–10k, 10k–100k or at least 100k tokens back, or never; the 1000 most frequent ids are left out of the earlier-chunk bins. The *locality gain* is the gain in reading order minus the gain on the same chunks shuffled. The *memory of A* is a snapshot's gain on A2 minus its gain on C2. *Collateral forgetting* is a snapshot's loss on a held-out set minus that of the pretrained model, averaged over the second half of the stream (positive means knowledge lost), and the *exchange rate* is the gain on the stream divided by the collateral forgetting on WikiText. Standard errors (SE) are computed over books (paired when two runs read the same chunks), or over the held-out chunks or books for collateral forgetting; a † marks a value under 2 SE, which we do not treat as a difference.

4. Results

4.1. A memory beyond the window, and it is local

With plain SGD at lr 10^{-4}, Pythia-14M reading the main set in order gains $+0.100 \pm 0.006$ nats/token over its frozen loss of 4.393 (Table 1). The gain falls with distance: $+0.307 \pm 0.011$ for rare tokens last seen less than 1k tokens back, $+0.144 \pm 0.008$ between 1k and 10k, and $+0.009 \pm 0.006^{\dagger}$ between 10k and 100k, while never-seen tokens lose -0.059 ± 0.013. Within a book the gain grows from $+0.013 \pm 0.008^{\dagger}$ in its first 1k tokens to $+0.139 \pm 0.008$ in its last 1k, while the shuffled learner's gain stays between +0.055 and +0.071 (Fig. 1b). Shuffling keeps 62% of the overall gain, but only 20% less than 1k tokens back and 24% between 1k and 10k (locality gain $+0.245 \pm 0.009$ and $+0.109 \pm 0.006$): most of the overall gain is adaptation to PG-19 text in general, and most of the long-range gain needs the order.

The memory is tied mostly to the current book. Splitting the earlier-chunk bins by whether the previous occurrence was in the same book, the locality gain between 1k and 10k tokens back is $+0.120 \pm 0.007$ for same-book tokens and $+0.035 \pm 0.009$ for earlier-book ones; between 10k and 100k, same-book tokens keep $+0.036 \pm 0.010$ and earlier-book tokens have none (-0.024 ± 0.006). It is also more than a count cache. A cache of recent token counts mixed into the frozen model ($p = 0.95\, p_{\text{model}} + 0.05\, p_{\text{cache}}$, counts halving every 1k tokens) is the better copier less than 1k tokens back ($+0.461 \pm 0.024$ against +0.307), but from 1k tokens on dynamic evaluation gains more ($+0.144$ against $+0.059 \pm 0.007$ between 1k and 10k; paired difference $+0.085 \pm 0.008$), and $+0.082 \pm 0.004$ more overall.

4.2. Locality as a variable

A cache should help more the longer the input stays on the same material. The gain falls at every step of decreasing locality (Fig. 2): $+0.176 \pm 0.023$ on the long books, $+0.100 \pm 0.006$ with $K = 1$, $+0.074 \pm 0.004$ with $K = 5$, $+0.065 \pm 0.003$ with $K = 25$ and $+0.062 \pm 0.003$ shuffled. On the main set the paired drops are -0.026 ± 0.002 from $K = 1$ to $K = 5$, -0.009 ± 0.002 from $K = 5$ to $K = 25$ and -0.003 ± 0.001 from $K = 25$ to shuffled, so interleaving only 5 books removes most of the in-order advantage. The long stream covers mostly different text, so the step from it to $K = 1$ is not paired.

4.3. Where the memory lives

We update one part of Pythia-14M at a time, each at its own learning rate. Freezing the output embedding costs little (-0.006 ± 0.001 overall against all parameters). The output embedding alone is a short-range cache: $+0.252 \pm 0.017$ less than 1k tokens back but $+0.072 \pm 0.006$ between 1k and 10k (same book), and the largest loss on never-seen tokens (-0.143 ± 0.012). Attention alone keeps less of the long-range gain ($+0.110 \pm 0.009$ between 1k and 10k, same book). Updating only the MLPs (lr 3×10^{-4}) gains more than updating all parameters: $+0.415 \pm 0.016$ against +0.307 less than 1k tokens back (paired difference $+0.108 \pm 0.008$) and $+0.107 \pm 0.007$ against +0.100 overall. One layer's MLP alone keeps between 36% and 93% of the full learner's gain less than 1k tokens back; the strongest is layer 5 and the last layer is the weakest (Fig. 3).

The MLP-only learner owes its larger gain to a stronger cache, not to better general adaptation: its locality gain is $+0.334 \pm 0.011$ against $+0.245 \pm 0.009$ less than 1k tokens back (paired $+0.088 \pm 0.005$), while shuffled it gains slightly less overall (-0.003 ± 0.001). It also forgets more WikiText ($+0.235 \pm 0.007$ against $+0.196 \pm 0.006$, paired $+0.039 \pm 0.003$), at a lower exchange rate (0.45 against 0.51). All parameters at its rate, 3×10^{-4}, are already worse on val-pick (4.2732 against 4.2565); with plain SGD at 14M attention takes 43% of the squared update norm, which may limit the step the full learner can use.

4.4. Eviction

If dynamic evaluation acts as a cache, reading other text should push a book out of memory (Fig. 4). With plain SGD on all parameters at lr 10^{-4}, the memory of A is $+0.097 \pm 0.013$ right after A1. It halves after about 6.1k tokens of other books and then declines slowly: $+0.044 \pm 0.007$ after 10k tokens, $+0.031 \pm 0.006$ after 100k and $+0.016 \pm 0.008$ after 300k. The snapshots never gain on the control book (-0.030 ± 0.007 right after A1, between -0.032 and -0.011 later), so the decline is the loss of A rather than general adaptation catching up. The memory is mostly about tokens that occurred in A1: rare tokens of A2 whose id occurs in A1 carry $+0.223 \pm 0.029$ after A1, other rare tokens $+0.056 \pm 0.011$; the former halve after about 2.9k tokens. A power law in I fits better than an exponential with a floor (χ^2 2.0 against 8.7, 4 degrees of freedom each), although the 7 points share the same 20 books.

Table 1. Gain over frozen Pythia-14M by distance to the token's previous occurrence (nats/token, ± SE over 25 books). Dynamic evaluation: plain SGD on all parameters, lr 10^{-4}, in reading order ($K = 1$) and shuffled; locality gain = in order − shuffled. Unigram cache: frozen model mixed with a decaying token-count cache; where the cache has no count for a token, mixing it in costs exactly $-\log 0.95 = 0.051$ nats (SE 0). †: under 2 SE.

bin (previous occurrence)	tokens	frozen loss	in order	shuffled	locality gain	unigram cache
all tokens	497,250	4.393	$+0.100_{\pm.006}$	$+0.062_{\pm.003}$	$+0.038_{\pm.004}$	$+0.018_{\pm.004}$
same chunk	9,886	5.006	$-0.111_{\pm.014}$	$-0.133_{\pm.010}$	$+0.022^{\dagger}_{\pm.013}$	$+0.096_{\pm.017}$
earlier, < 1k	17,036	7.335	$+0.307_{\pm.011}$	$+0.062_{\pm.007}$	$+0.245_{\pm.009}$	$+0.461_{\pm.024}$
earlier, 1k-10k	33,973	6.997	$+0.144_{\pm.008}$	$+0.035_{\pm.005}$	$+0.109_{\pm.006}$	$+0.059_{\pm.007}$
earlier, 10k-100k	44,109	7.113	$+0.009^{\dagger}_{\pm.006}$	$+0.026_{\pm.005}$	$-0.017_{\pm.006}$	$-0.051_{\pm.000}$
earlier, ≥ 100k	13,125	7.824	$-0.039_{\pm.009}$	$-0.017_{\pm.008}$	$-0.023_{\pm.008}$	$-0.051_{\pm.000}$
never seen	20,976	8.190	$-0.059_{\pm.013}$	$-0.055_{\pm.009}$	$-0.004^{\dagger}_{\pm.010}$	$-0.051_{\pm.000}$

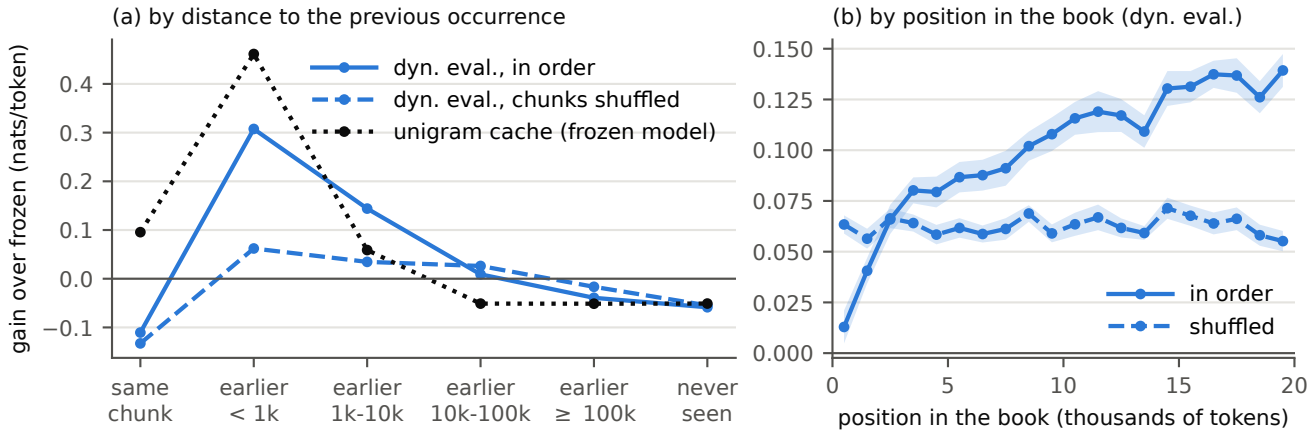


Fig. 1. Gain over frozen Pythia-14M with plain SGD (lr 10^{-4}). (a) By distance to the token's previous occurrence, for dynamic evaluation in order and shuffled and for the unigram cache. (b) By position in the book, mean over the 24 books that follow a book switch, ±1 SE over books.

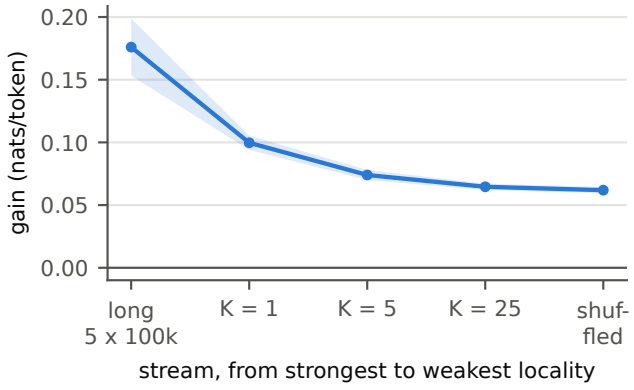


Fig. 2. Gain over frozen Pythia-14M (plain SGD, all parameters, lr 10^{-4}, ±1 SE over books) on streams of decreasing locality: 5 long books of 100k tokens, then the main set with books in order ($K = 1$), 5 or 25 books interleaved, and shuffled.

4.5. Collateral forgetting

A cache should leave the rest of memory intact. Reading the main set in order, Pythia-14M (plain SGD, lr 10^{-4}) forgets $+0.196 \pm 0.006$ nats/token on WikiText (pretrained loss 4.633), about 2.0 times its gain on the stream (exchange rate 0.51), but loses nothing on the held-out PG-19 books ($-0.006 \pm 0.010^{\dagger}$). The shuffled learner forgets less on WikiText ($+0.112 \pm 0.004$) and improves on the unseen books (-0.045 ± 0.008): the part of the gain that survives shuffling is adaptation to PG-19 text, and it carries over. Reading in order forgets more than shuffled on both sets (-0.084 ± 0.003 on WikiText, -0.040 ± 0.002 on the books), at about the same exchange rate (0.51 and 0.55). Over the very long stream (800k tokens), forgetting on WikiText averages +0.143 over the snapshots in the first book and +0.273, +0.269 and +0.267 in the next three: it levels off after the first book, while the held-out books lose a little ($+0.052 \pm 0.013$).

4.6. Update rules, model size and a trade-off

We compare three update rules at three sizes, each at its own learning rate (Table 2). Over the first 40 chunks, plain SGD moves the MLP weights by 1.2×10^{-4} of their norm per step at 14M and 1.1×10^{-6} at 70M, about 110 times less, and at 70M attention takes 78% of the squared update norm. With plain SGD, Pythia-70M has almost no locality gain ($+0.016 \pm 0.003$ less than 1k tokens back). Normalized SGD more than doubles 70M's gain (paired difference $+0.033 \pm 0.002$) and gives it some locality gain ($+0.084 \pm 0.006$), far less than at 14M and 31M ($+0.372 \pm 0.011$ and $+0.418 \pm 0.013$), which are the largest of any rule. Adam has no cache at any size (locality gain less than 1k tokens back -0.052 ± 0.009, -0.070 ± 0.011 and -0.140 ± 0.010). From 14M to 31M, plain SGD's locality gain does not change beyond noise ($+0.017 \pm 0.014^{\dagger}$ less than 1k tokens back), and the smaller model does not pay more general knowledge per nat of gain.

If the cache and general knowledge share the same weights, filling one should erode the other. Placing every run with snapshots on these two axes (Fig. 5), the order within 14M is the same on both: Adam (-0.052, $+0.141$), plain SGD on

Table 2. Three update rules at three sizes, all parameters, each at its val-pick learning rate: gain with $K = 1$ over each model's frozen loss and locality gain (in order − shuffled, paired), ± SE over books; collateral forgetting on WikiText ($K = 1$, second half of the stream, ± SE over chunks); exchange rate (stream gain / collateral forgetting on WikiText) in order and shuffled. †: under 2 SE.

size	rule	lr	gain, $K = 1$	loc. gain, < 1k	loc. gain, all	collateral forgetting, WikiText	exch. rate, $K = 1$	shuffled
14M	plain SGD	10^{-4}	$+0.100_{\pm.006}$	$+0.245_{\pm.009}$	$+0.038_{\pm.004}$	$+0.196_{\pm.006}$	0.51	0.55
	Adam	3×10^{-6}	$+0.086_{\pm.005}$	$-0.052_{\pm.009}$	$-0.004^{\dagger}_{\pm.004}$	$+0.141_{\pm.005}$	0.61	0.91
	normalized SGD	10^{-4}	$+0.113_{\pm.008}$	$+0.372_{\pm.011}$	$+0.056_{\pm.006}$	$+0.316_{\pm.008}$	0.36	0.29
31M	plain SGD	3×10^{-4}	$+0.085_{\pm.005}$	$+0.262_{\pm.010}$	$+0.040_{\pm.004}$	$+0.228_{\pm.005}$	0.37	0.31
	Adam	3×10^{-6}	$+0.070_{\pm.004}$	$-0.070_{\pm.011}$	$-0.007_{\pm.003}$	$+0.137_{\pm.004}$	0.51	0.75
	normalized SGD	10^{-4}	$+0.092_{\pm.007}$	$+0.418_{\pm.013}$	$+0.060_{\pm.005}$	$+0.348_{\pm.006}$	0.26	0.13
70M	plain SGD	10^{-5}	$+0.025_{\pm.002}$	$+0.016_{\pm.003}$	$+0.004_{\pm.002}$	$+0.010_{\pm.002}$	2.57	2.67
	Adam	3×10^{-7}	$+0.029_{\pm.002}$	$-0.140_{\pm.010}$	$-0.015_{\pm.002}$	$+0.060_{\pm.003}$	0.47	0.68
	normalized SGD	3×10^{-6}	$+0.057_{\pm.004}$	$+0.084_{\pm.006}$	$+0.018_{\pm.003}$	$+0.039_{\pm.002}$	1.48	1.52

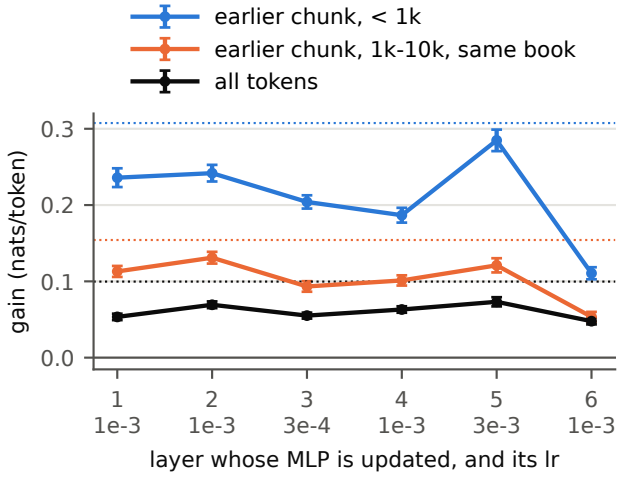


Fig. 3. Gain over frozen Pythia-14M with plain SGD when only one layer's MLP is updated, each at its own learning rate (below the layer number), ±1 SE over books. Dotted lines: all parameters at lr 10^{-4}.

all parameters (+0.245, +0.196), plain SGD on the MLPs only (+0.334, +0.235) and normalized SGD (+0.372, +0.316). The same holds within 31M, and at 70M for plain and normalized SGD, but Adam at 70M forgets more than both (+0.060±0.003) with negative locality gain. Across sizes the points do not lie on one curve: at 70M every rule forgets far less for a given locality gain. With ten points, one seed and one model family, this is a consistent ordering within the settings we tried, not a law.

4.7. Long books and model size

A cache is worth more the longer the material stays the same, and a small model might use it to approach a larger one (Table 3). On the very long stream, Pythia-14M with plain SGD (lr 10^{-4}) gains +0.170 ± 0.011 over its frozen loss and closes 49% of the gap to a frozen Pythia-31M. The share grows from 7% in the first 10k tokens of a book to 79% between 160k and 170k tokens, and is 56% in the last 10k; frozen 31M stays ahead overall (+0.180 ± 0.019). Pythia-31M with dynamic evaluation (lr 3×10^{-4}) gains about as much (+0.168±0.013 over its frozen loss), stays ahead of 14M with dynamic evaluation, and does not reach a frozen Pythia-70M (+0.054 ± 0.010 behind).

Table 3. Four PG-19 books of 200k tokens, read book after book: loss, gain over frozen Pythia-14M (± SE over 4 books, paired) and share of the gap between frozen 14M and frozen 31M closed. Dynamic evaluation: plain SGD on all parameters.

run	loss	gain over frozen 14M	share of 14M-31M gap
frozen 14M	4.419		
14M dyn. eval., lr 10^{-4}	4.249	$+0.170_{\pm.011}$	49%
frozen 31M	4.069	$+0.351_{\pm.010}$	100%
31M dyn. eval., lr 3×10^{-4}	3.901	$+0.518_{\pm.013}$	148%
frozen 70M	3.846	$+0.573_{\pm.013}$	163%

5. Discussion and limitations

Dynamic evaluation helps more when the input has locality: the gain from more than 1k tokens back mostly disappears when the chunks are shuffled, and the overall gain falls at every step from long books to shuffled chunks. The memory it builds is evicted as a cache's would be, halving after a few thousand tokens of other books with a long tail. Unlike a cache, however, filling it overwrites general knowledge outside the domain, about twice the gain on the stream, and within 14M and 31M every setting we tried moved cache and collateral forgetting together, so no knob tested here separates the two. The effect is not larger in the smaller model, at least between 14M and 31M, and 70M has less cache, depending on the update rule and the step it allows. Nor can a small model with dynamic evaluation replace a larger one at these sizes: on long books it is worth about half the step from 14M to 31M, but 31M gains as much from it.

Our experiments are deliberately small: a single seed of three sizes of one model family reading PG-19, with WikiText as the out-of-domain reference. Whether the behaviour holds in larger models and on other inputs with locality, such as code bases and long conversations, remains to be tested, and PolyPythias (van der Wal et al., 2025) would allow repeating the analysis across pretraining seeds. Scoring each chunk without earlier context isolates the memory in the weights; a practical assessment needs baselines with the full context window and with matched compute. Since the cache and general knowledge compete for the same weights, a separate fast part on top of a frozen or slowly learning model could hold the recent context, and ablating Adam's momentum and per-weight normalization could show which property of the update rule a cache in the weights depends on.

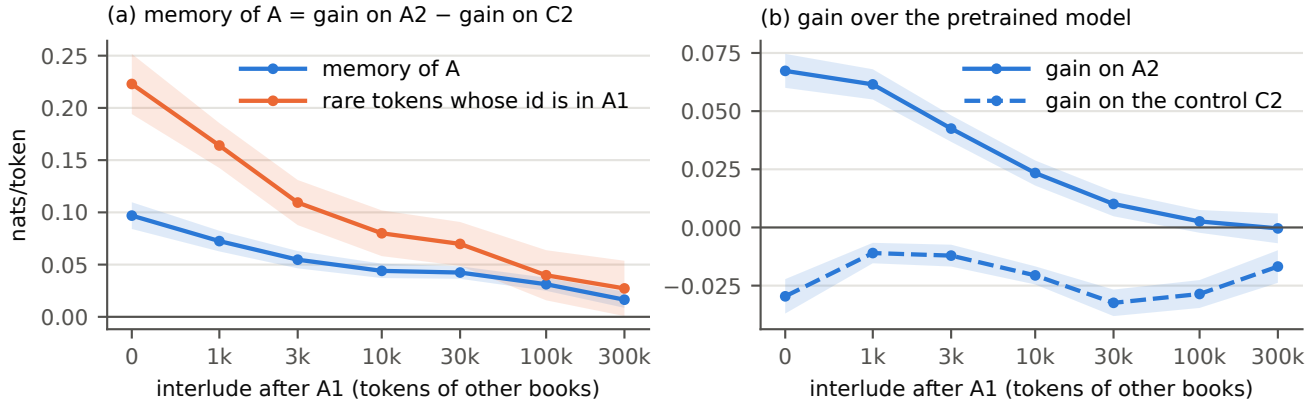


Fig. 4. Eviction probed with frozen snapshots (Pythia-14M, plain SGD, all parameters, lr 10^{-4}; mean of 20 books, bands ±1 SE over books), against the number of tokens of other books read after A1. (a) Memory of A (gain on A2 − gain on the never-read control C2, same snapshot), and the same for rare tokens whose id occurs in A1. (b) Gain over the pretrained model on A2 and on C2.

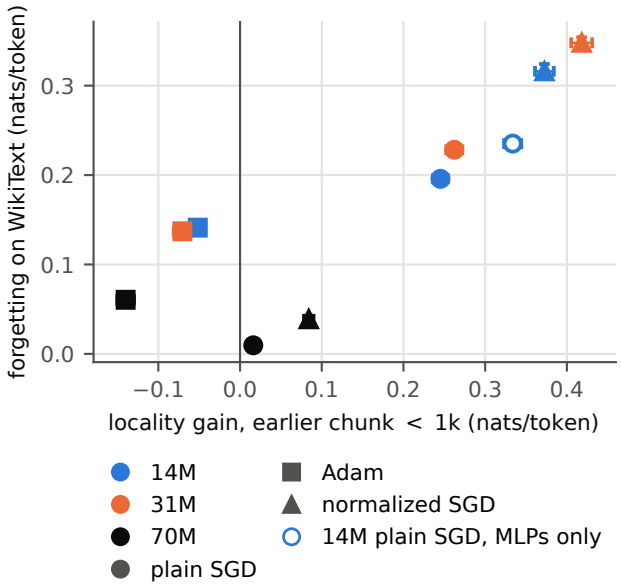


Fig. 5. Locality gain less than 1k tokens back against collateral forgetting on WikiText ($K = 1$, second half of the stream) for every run with snapshots: three update rules (marker) at three sizes (colour) on all parameters, and plain SGD on the MLPs only at 14M (open circle), each at its own learning rate. Bars: ±1 SE.

6. Conclusion

A standard transformer updated by plain gradient steps has a cache in its weights: its benefit depends on locality (only 20% of the gain less than 1k tokens back survives shuffling), it is tied to the book being read, it sits mostly in the MLPs, and other text evicts it after about 6.1k tokens. It is not a clean cache: knowledge of the same domain is preserved, but filling the cache costs about 2.0 nats of WikiText loss per nat gained. Even so, on long books a 14M-parameter model with dynamic evaluation closes 49% of the gap to a frozen model about twice its size. Separating eviction from collateral forgetting could strengthen the cache and move towards small models that perform on the context at hand like larger ones.

Acknowledgements

This work has been financed by Project PID2024-160996OB-I00, under grant agreement MICIU/AEI/10.13039/501100011033 and FEDER, UE, and by Generalitat de Catalunya (AGAUR) under grant agreement 2021-SGR-00478.

Data availability

The code, the experimental results and the scripts that generate the tables and figures are available at `https://github.com/rtous/goldfish`.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist with editing the text and preparing the manuscript for submission. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Anderson, J.R., Schooler, L.J., 1991. Reflections of the environment in memory. Psychological Science 2, 396–408.

Ba, J., Hinton, G.E., Mnih, V., Leibo, J.Z., Ionescu, C., 2016. Using fast weights to attend to the recent past, in: Advances in Neural Information Processing Systems 29 (NIPS).

Bansal, R., Zhang, A., Tiwari, R., Madaan, L., Duvvuri, S.S., Khatri, D., Brandfonbrener, D., Alvarez-Melis, D., Bhargava, P., Kale, M.S., Jelassi, S., 2026. Let's (not) just put things in context: Test-time training for long-context LLMs, in: International Conference on Learning Representations (ICLR). ArXiv:2512.13898.

Behrouz, A., Zhong, P., Mirrokni, V., 2024. Titans: Learning to memorize at test time. arXiv:2501.00663.

Biderman, S., Bicheno, K., Gao, L., 2022. Datasheet for the Pile. arXiv:2201.07311.

Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M.A., Purohit, S., Prashanth, U.S., Raff, E., Skowron, A., Sutawika, L., van der Wal, O., 2023. Pythia: A suite for analyzing large language models across training and scaling, in: Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 2397–2430.

Bulatov, A., Kuratov, Y., Burtsev, M., 2022. Recurrent memory transformer, in: Advances in Neural Information Processing Systems (NeurIPS).

Chang, H., Park, J., Ye, S., Yang, S., Seo, Y., Chang, D.S., Seo, M., 2024. How do large language models acquire factual knowledge during pretraining?, in: Advances in Neural Information Processing Systems (NeurIPS).

Church, K.W., 2000. Empirical estimates of adaptation: The chance of two Noriegas is closer to $p/2$ than p^2, in: Proceedings of the 18th International Conference on Computational Linguistics (COLING), pp. 180–186.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q.V., Salakhutdinov, R., 2019. Transformer-XL: Attentive language models beyond a fixed-length context, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 2978–2988.

Denning, P.J., 1968. The working set model for program behavior. Communications of the ACM 11, 323–333.

Feng, G., Luo, S., Hua, K., Zhang, G., He, D., Huang, W., Cai, T., 2026. In-place test-time training, in: International Conference on Learning Representations (ICLR). ArXiv:2604.06169.

French, R.M., 1999. Catastrophic forgetting in connectionist networks. Trends in Cognitive Sciences 3, 128–135.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., Leahy, C., 2020. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027.

Geva, M., Schuster, R., Berant, J., Levy, O., 2021. Transformer feed-forward layers are key-value memories, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 5484–5495.

Grave, E., Joulin, A., Usunier, N., 2017. Improving neural language models with a continuous cache, in: International Conference on Learning Representations (ICLR).

Graves, A., 2013. Generating sequences with recurrent neural networks. arXiv:1308.0850.

Hinton, G.E., Plaut, D.C., 1987. Using fast weights to deblur old memories, in: Proceedings of the Ninth Annual Conference of the Cognitive Science Society, Erlbaum, Hillsdale, NJ. pp. 177–186.

Jelinek, F., Merialdo, B., Roukos, S., Strauss, M., 1991. A dynamic language model for speech recognition, in: Speech and Natural Language: Proceedings of a Workshop Held at Pacific Grove, California, pp. 293–295.

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., Lewis, M., 2020. Generalization through memorization: Nearest neighbor language models, in: International Conference on Learning Representations (ICLR).

Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization, in: International Conference on Learning Representations.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., Hadsell, R., 2017. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences 114, 3521–3526.

Krause, B., Kahembwe, E., Murray, I., Renals, S., 2018. Dynamic evaluation of neural sequence models, in: Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 2766–2775.

Krause, B., Kahembwe, E., Murray, I., Renals, S., 2019. Dynamic evaluation of transformer language models. arXiv:1904.08378.

Kuhn, R., De Mori, R., 1990. A cache-based natural language model for speech recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence 12, 570–583.

Lazaridou, A., Kuncoro, A., Gribovskaya, E., Agrawal, D., Liska, A., Terzi, T., Gimenez, M., de Masson d'Autume, C., Kocisky, T., Ruder, S., Yogatama, D., Cao, K., Young, S., Blunsom, P., 2021. Mind the gap: Assessing temporal generalization in neural language models, in: Advances in Neural Information Processing Systems (NeurIPS).

Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., Chandra, V., 2024. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases, in: Proceedings of the 41st International Conference on Machine Learning (ICML), pp. 32431–32454.

Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., Zhang, Y., 2023. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. arXiv:2308.08747.

McCloskey, M., Cohen, N.J., 1989. Catastrophic interference in connectionist networks: The sequential learning problem, in: Psychology of Learning and Motivation. Academic Press. volume 24, pp. 109–165.

Meng, K., Bau, D., Andonian, A., Belinkov, Y., 2022. Locating and editing factual associations in GPT, in: Advances in Neural Information Processing Systems (NeurIPS).

Merity, S., Xiong, C., Bradbury, J., Socher, R., 2017. Pointer sentinel mixture models, in: International Conference on Learning Representations.

Mikolov, T., Karafiát, M., Burget, L., Černocký, J., Khudanpur, S., 2010. Recurrent neural network based language model, in: Proceedings of Interspeech, pp. 1045–1048.

Rae, J.W., Potapenko, A., Jayakumar, S.M., Hillier, C., Lillicrap, T.P., 2020. Compressive transformers for long-range sequence modelling, in: International Conference on Learning Representations (ICLR).

Rannen-Triki, A., Bornschein, J., Pascanu, R., Hutter, M., György, A., Galashov, A., Teh, Y.W., Titsias, M.K., 2024. Revisiting dynamic evaluation: Online adaptation for large language models. arXiv:2403.01518.

Schmidhuber, J., 1992. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. Neural Computation 4, 131–139.

Sun, Y., Li, X., Dalal, K., et al., 2024. Learning to (learn at test time): RNNs with expressive hidden states. arXiv:2407.04620.

Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A.A., Hardt, M., 2020. Test-time training with self-supervision for generalization under distribution shifts, in: Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 9229–9248.

Tandon, A., Dalal, K., Li, X., Koceja, D., Rød, M., Buchanan, S., Wang, X., Leskovec, J., Koyejo, S., Hashimoto, T., Guestrin, C., McCaleb, J., Choi, Y., Sun, Y., 2025. End-to-end test-time training for long context. arXiv:2512.23675.

Tu, Z., Su, Z., Devanbu, P., 2014. On the localness of software, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), pp. 269–280.

van der Wal, O., Lesci, P., Müller-Eberstein, M., Saphra, N., Schoelkopf, H., Zuidema, W., Biderman, S., 2025. PolyPythias: Stability and outliers across fifty language model pre-training runs, in: International Conference on Learning Representations (ICLR).

Wu, Y., Rabe, M.N., Hutchins, D., Szegedy, C., 2022. Memorizing transformers, in: International Conference on Learning Representations (ICLR).